

Implementation Plan

PU PS 6: Intelligent Meeting & Action Tracking System (Simplified MVP)

This document outlines a streamlined implementation plan for a functional prototype. This approach prioritizes a fast, visually compelling front-end demonstration of the core requirements without the overhead of a complex backend infrastructure or real authentication.

Core Objective

Build a lightweight web application that demonstrates the end-to-end flow of scheduling a meeting, generating AI-extracted summaries and tasks from a transcript, and tracking those tasks.

1. Application Flow & Key Pages

Page 1: Simplified Login

- **Purpose:** Demonstrate role-based access control.
- **Implementation:** A simple UI with two entry paths: "Log in as Organizer" and "Log in as Participant."
- **Logic:** Stores the chosen role in the browser's local state to dictate whether edit controls are visible on subsequent pages.

Page 2: Main Dashboard

- **Purpose:** A centralized view of the user's responsibilities and schedule.
- **Components:**
 - **Upcoming Meetings:** A list of scheduled meetings.
 - **Pending Tasks:** Action items assigned to the current user that are not yet complete.
 - **Completed Tasks:** A historical view of finished action items.
- **Actionable Items:** Each pending task will feature a manual "Send Reminder" button that triggers a UI notification, simulating the escalation workflow.

Page 3: Add Meeting Form

- **Purpose:** Capture meeting logistics.
- **Fields:** Meeting Title, Date & Time, Participants (multi-select or comma-separated list).

- **Logic:** Upon submission, the meeting data is saved to browser `localStorage` and updates the Dashboard view.

Page 4: Meeting Details & AI Extraction

- **Purpose:** The core "intelligent" feature demonstrating automated record conversion.
- **Implementation:** A large text input area designed to accept a pasted text transcript of a meeting (bypassing live audio processing for the demo), and a "Generate Insights" button.
- **AI Integration:** The frontend will send the pasted transcript to an LLM API (e.g., Gemini or Groq) with a strict prompt to return a structured response containing:
 1. A general meeting summary.
 2. A list of formal decisions made.
 3. A structured list of tasks (Action Item, Assignee, Deadline).
- **Display & Permissions:** The results are rendered on the page. Based on the login role (from Page 1), only the Organizer will see edit/delete buttons next to the generated tasks.

2. Technical Stack (Simplified)

- **Frontend Framework:** Next.js (React) using Tailwind CSS for rapid styling and a clean, modern UI.
- **State Management:** React Context or simple browser `localStorage` to handle data persistence between page reloads (bypassing the need for a PostgreSQL/Prisma setup).
- **AI Integration:** Direct frontend-to-API calls using an LLM provider (like Groq for fast inference or the Gemini API).
- **Mock Data:** Pre-populate the dashboard with realistic dummy data to ensure the application looks populated during the initial demo.

3. Development Milestones

Milestone 1: Scaffold UI

Build the visual layout for the Login, Dashboard, and Add Meeting pages using mock data.

Milestone 2: State Management

Implement `localStorage` so adding a meeting updates the Dashboard. Implement role switching (Organizer vs Participant).

Milestone 3: AI Integration

Build the Meeting Details page. Write the prompt engineering logic. Connect the UI to the LLM API to process a sample text transcript.

Milestone 4: Polish & Refine

Add conditional rendering (hiding edit buttons from participants). Implement the UI pop-ups for the "Send Reminder" buttons.